

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 24.Б42-мм

Ускорение алгоритма gSpan в Desbordante

Малышев Максим Владимирович

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
ассистент кафедры ИАС, к. ф.-м. н., Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Задача поиска частых подграфов	5
2.2. Алгоритм gSpan	6
2.3. Вычислительная сложность и узкие места	8
2.4. Методы оптимизации	8
3. Описание решения	10
3.1. Профилирование	10
3.2. Оптимизация структур данных и устранение аллокаций	11
3.3. Алгоритмические оптимизации и параллелизация	13
4. Эксперимент	19
4.1. Однопоточное сравнение	19
4.2. Исследование многопоточности	20
4.3. Потребление памяти	21
4.4. Накладные расходы Python	22
Заключение	25
Список литературы	26

Введение

В современном мире объемы графовых данных растут экспоненциально, что делает скорость их анализа критически важным фактором для практического применения. Графовые модели данных находят применение в самых разных вычислительно сложных областях: от анализа молекулярных структур [4] до масштабных социальных сетей [3] и программного кода [1]. Одной из ресурсоёмких задач в этой сфере является поиск частых подграфов (Frequent Subgraph Mining, FSM). Эта задача заключается в выявлении подструктур, которые встречаются в наборе графов чаще определённого порога. Однако при росте пространства поиска время вычислений резко возрастает, и алгоритмы начинают работать слишком долго.

Высокая вычислительная сложность задачи, обусловленная проблемой изоморфизма подграфов, требует применения высокоэффективных алгоритмов, таких как gSpan [5]. Однако даже такой алгоритм может работать слишком долго, если его программная реализация выполнена без детальной оптимизации кода и рационального использования системных ресурсов.

Desbordante [2] — высокопроизводительная C++ библиотека для профилирования данных и поиска закономерностей. Ключевым требованием к разрабатываемым компонентам библиотеки является максимальное быстродействие при обработке больших объёмов информации, поэтому минимизация времени работы алгоритмов выступает обязательным условием для успешного расширения её функциональности.

Данная работа продолжает предыдущую [6], в рамках которой базовая версия алгоритма gSpan была интегрирована¹ в библиотеку Desbordante. Однако её первая реализация на C++ уступала по времени выполнения существующим аналогам на некоторых тестовых наборах данных. Оптимизация алгоритма gSpan призвана устранить эти вычислительные издержки, обеспечив необходимую масштабируемость графового модуля библиотеки.

¹<https://github.com/Desbordante/desbordante-core/pull/666>

1. Постановка задачи

Целью данной работы является ускорение алгоритма gSpan в библиотеке Desbordante. Для этого были поставлены следующие задачи:

- провести профилирование текущей реализации для поиска узких мест и устранить их;
- изучить сторонние оптимизированные решения и перенести эффективные подходы в проект Desbordante;
- сравнить производительность оптимизированного алгоритма с предыдущей версией.

2. Обзор

Теоретические основы алгоритма gSpan подробно описаны в предыдущем отчёте [6] за осень 2025 года. В данном разделе кратко напомним ключевые понятия и приводится базовый псевдокод алгоритма, который необходим для последующего анализа его производительности.

2.1. Задача поиска частых подграфов

Пусть задан коллекция графов² $D = \{G_1, G_2, \dots, G_n\}$, состоящая из n связных помеченных графов. Каждый граф $G_i = (V_i, E_i, L_i, l_i)$ представляет собой четвёрку: множество вершин V_i , множество рёбер $E_i \subseteq V_i \times V_i$, множество меток L_i и функция разметки $l_i : V_i \cup E_i \rightarrow L_i$.

Требуется найти все связные помеченные подграфы g , для которых поддержка (support) в коллекции графов D не меньше заданного порога min_sup :

$$support(g) = |\{G_i \in D \mid g \sqsubseteq G_i\}| \geq min_sup,$$

где $g \sqsubseteq G_i$ означает, что подграф g изоморфен некоторому подграфу G_i .

Для устранения дубликатов при поиске gSpan использует представление подграфов в виде канонических **DFS-кодов** (последовательностей рёбер, упорядоченных в порядке обхода графа в глубину). Задача поиска частых подграфов сводится к генерации и проверке только **минимальных** DFS-кодов, что доказано теоремой об изоморфизме [5]:

$$G_1 \cong G_2 \iff min(Z(G_1)) = min(Z(G_2)).$$

Подробные правила построения DFS-деревьев и лексикографического порядка кодов приведены в работе [6].

²В оригинальной статье [5] используется термин «база данных графов» (*graph database*), однако во избежание путаницы с системами управления базами данных (СУБД) далее используется более точный термин «коллекция графов».

2.2. Алгоритм gSpan

Алгоритм gSpan не генерирует лишних кандидатов, а последовательно наращивает частые подграфы на основе уже найденных частых фрагментов. Он состоит из двух основных процедур.

2.2.1. Основная процедура: GraphSet_Projection

Процедура 1 выполняет предварительную фильтрацию редких рёбер и вершин, строит набор однорёберных «семян» ($\mathbb{S}^1$) и запускает рекурсивный поиск.

Algorithm 1 GraphSet_Projection($\mathbb{D}, \mathbb{S}$)

Input: A graph database $\mathbb{D}$, minimum support threshold $minSup$.

Output: A set of frequent subgraphs $\mathbb{S}$.

- 1: sort the labels in $\mathbb{D}$ by their frequency;
 - 2: remove infrequent vertices and edges;
 - 3: relabel the remaining vertices and edges;
 - 4: $\mathbb{S}^1 \leftarrow$ all frequent 1-edge graphs in $\mathbb{D}$;
 - 5: sort $\mathbb{S}^1$ in DFS lexicographic order;
 - 6: $\mathbb{S} \leftarrow \mathbb{S}^1$;
 - 7: **for** each edge $e \in \mathbb{S}^1$ **do**
 - 8: initialize s with e , set $s.D$ by graphs which contains e ;
 - 9: Subgraph_Mining($\mathbb{D}, \mathbb{S}, s$);
 - 10: $\mathbb{D} \leftarrow \mathbb{D} - e$;
 - 11: **if** $|\mathbb{D}| < minSup$ **then**
 - 12: **break**;
 - 13: **end if**
 - 14: **end for**
-

Шаги 1–6 выполняют предобработку: удаляются редкие метки, которые не могут входить в частые подграфы, а оставшиеся метки перенумеровываются для обеспечения лексикографического порядка. Множество $\mathbb{S}^1$ частых однорёберных подграфов служит набором «семян» (seeds) для последующего роста.

Шаги 7–14 реализуют основной цикл поиска. Для каждого семени e формируется его проекция $s.D$ — подмножество графов из $\mathbb{D}$, содержащих ребро e . Затем вызывается рекурсивная процедура **Subgraph_Mi-**

ning, которая строит всё поддереву дерева DFS-кодов, корнем которого является e . После обработки поддерева ребро e удаляется из всех графов в $\mathbb{D}$ (проекция набора данных), что сокращает размер данных для последующих итераций.

2.2.2. Рекурсивная процедура: Subgraph_Mining

Процедура 2 выполняет рекурсивный рост кодов в глубину дерева DFS-кодов.

Algorithm 2 Subgraph_Mining($\mathbb{D}, \mathbb{S}, s$)

Input: A graph database $\mathbb{D}$, a set of frequent subgraphs $\mathbb{S}$, a candidate subgraph s , and a threshold $minSup$.
Output: An updated set of frequent subgraphs $\mathbb{S}$.

```

1: if  $s \neq min(s)$  then
2:   return;
3: end if
4:  $\mathbb{S} \leftarrow \mathbb{S} \cup \{s\}$ ;
5: enumerate  $s$  in each graph in  $\mathbb{D}$  and count its children;
6: for each  $c, c$  is  $s$ ' child do
7:   if  $support(c) \geq minSup$  then
8:      $s \leftarrow c$ ;
9:     Subgraph_Mining( $\mathbb{D}_s, \mathbb{S}, s$ );
10:  end if
11: end for

```

Процедура проверяет, является ли текущий DFS-код s минимальным (строка 1). Если нет, подграф отсекается как дубликат. В противном случае s добавляется в множество результатов (строка 4). Затем для каждого графа в проекции $\mathbb{D}$ перечисляются все вхождения s и подсчитывается поддержка его потомков (строка 5). Для каждого потомка c с достаточной поддержкой рекурсивно вызывается **Subgraph_Mining** (строки 6–11), продолжая рост подграфа.

2.3. Вычислительная сложность и узкие места

Хотя с теоретической точки зрения gSpan является эффективным алгоритмом, на практике он сталкивается с серьёзными вычислительными ограничениями. Ключевыми узкими местами, снижающими производительность его базовой реализации, являются следующие:

1. **Проверка минимальности DFS-кода** (строка 1 в алгоритме 2). Процедура $s \neq \min(s)$ отсекает дубликаты кандидатов, но требует построение минимального кода. В текущей реализации это выполняется «в лоб»: на каждой проверке с нуля строится отдельный граф и для каждого ребра генерируются вообще все возможные варианты его расширения, что создаёт постоянные накладные расходы.
2. **Поиск вхождений подграфа** (строка 5 в алгоритме 2). Перечисление расширений текущего кода s во всей базе данных $\mathbb{D}$ требует многократного решения задачи изоморфизма подграфов. Без использования специальных структур данных этот шаг выполняется путём повторного сканирования исходных графов и отнимает основную долю процессорного времени.
3. **Интенсивная работа с памятью.** Рекурсивный характер gSpan и постоянное создание временных объектов (DFS-кодов, рёбер, проекций баз данных) приводят к частым операциям выделения и удаления памяти. Без специальной оптимизации такие постоянные обращения к памяти существенно замедляют работу программы.

2.4. Методы оптимизации

Среди методов оптимизации алгоритма можно выделить три ключевых:

1. **Низкоуровневые оптимизации и управление памятью.** Процесс поиска частых подграфов требует интенсивной работы с ди-

намической памятью в рекурсивных вызовах под создание временных DFS-кодов, списков кандидатов и вспомогательных контейнеров. В связи с этим критически важной становится низкоуровневая оптимизация кода. Это включает в себя замену стандартных контейнеров C++ на более компактные структуры данных, уменьшение числа выделений памяти в куче (heap) при частых операциях, предварительное выделение буферов, а также улучшение локальности данных в кэше процессора.

2. Алгоритмические оптимизации и кэширование. Одним из способов ускорения строки 5 в алгоритме 2 является сохранение и кэширование информации о вхождении подграфов (embeddings). Вместо того чтобы на каждом шаге рекурсии осуществлять повторный поиск подграфа во всей коллекции (например, через обход графа с помощью вспомогательных отображений меток на вершины), алгоритм может сохранять списки уже найденных рёбер текущего подграфа. Это позволяет искать новые расширения только среди смежных с ними рёбер, значительно сокращая число проходов по графу. Также на этом этапе применяются эффективные эвристики для раннего отсечения неперспективных кандидатов.

3. Параллельные вычисления. Пространство поиска в алгоритме gSpan образует дерево DFS-кодов, в котором обработка различных поддеревьев не зависит друг от друга. Это позволяет эффективно применять параллелизм на уровне задач, динамически распределяя ветви рекурсивного поиска между ядрами процессора с помощью пулов потоков или специализированных библиотек (например, OpenMP, Intel TBB или `std::thread`).

В данной работе использовался совмещённый подход. Сначала с помощью утилиты `perf` были найдены самые узкие места в исходной программе. Затем были изучены более быстрые открытые реализации gSpan, а их решения — перенесены в Desbordante.

3. Описание решения

3.1. Профилирование

Перед началом работ по оптимизации было проведено измерение производительности исходной реализации алгоритма gSpan. На тестовом графе `mutag` (см. Таблицу 1) при пороге поддержки $min_sup = 0.5$ алгоритм показал крайне низкую скорость работы — время выполнения составило ≈ 32 с.

Для выявления причин медленной работы было проведено профилирование программы с помощью утилиты `perf`. Анализ построенной диаграммы вызовов позволил выявить два ключевых узких места:

1. **Выделение и освобождение динамической памяти.** Значительная часть времени тратится на системные вызовы выделения и освобождения памяти (`malloc` и `free`) внутри глубоких рекурсивных вызовов функции майнинга. Это вызвано постоянным динамическим созданием временных контейнеров (`std::vector`, `std::map` и др.) под каждый проверяемый кандидат на каждом шаге рекурсии, а также их избыточным копированием при передаче данных между функциями вместо использования ссылок или семантики перемещения.
2. **Неоптимальный поиск вхождений подграфа.** Функция поиска изоморфизма занимала основную долю времени вычислений. В исходной реализации при добавлении каждого нового ребра поиск вхождений подграфа фактически выполнялся заново путём повторного обхода коллекции графов, без кэширования промежуточных результатов. Кроме того, для хранения соответствий вершин использовались ассоциативные контейнеры, создававшие дополнительные накладные расходы при постоянных обращениях к элементам.

Результаты профилирования показали, что для ускорения алгоритма необходимо внедрить кэширование вхождений вместо их повторного

поиска с нуля, перейти к более компактным структурам данных, устранить лишнее копирование, а также минимизировать динамические аллокации памяти внутри рекурсивных вызовов.

3.2. Оптимизация структур данных и устранение аллокаций

На основе данных профилирования была проведена серия последовательных улучшений исходного кода.

3.2.1. Отказ от ассоциативных контейнеров и локальные оптимизации

В исходной реализации тип `std::vector<std::unordered_map<int, vertex_t>>` применялся для хранения найденных изоморфизмов. В рамках первого этапа оптимизации были выполнены следующие изменения:

- **Внедрение семантики перемещения.** Добавление операторов `std::move` при формировании списков отображений исключило глубокое копирование промежуточных контейнеров, сократив время работы до ≈ 20 с.
- **Переход к индексным массивам.** Поскольку идентификаторы вершин подграфа последовательны и начинаются с нуля, структура хранения изоморфизмов была заменена на `std::vector<std::vector<vertex_t>>`. Номер вершины в DFS-коде стал прямым индексом во внутреннем векторе. Такое решение полностью устранило накладные расходы на хэширование и снизило время работы до ≈ 7 секунд.
- **Вынос аллокаций из циклов.** Вспомогательный вектор обратных отображений размера n (число вершин графа) ранее создавался заново на каждой итерации цикла по найденным изоморфизмам. Создание этого вектора было вынесено за пределы цикла с предварительным резервированием памяти. Внутри цикла

память больше не выделялась: после обработки каждого изоморфизма изменённые элементы точно сбрасывались в исходное состояние по известным индексам. В результате время выполнения сократилось до ≈ 5 секунд.

3.2.2. Переиспользование вектора изоморфизмов

Несмотря на отказ от хэш-таблиц, функция поиска изоморфизмов при каждом рекурсивном спуске продолжала выделять память под новые векторы отображений. Для устранения этих постоянных аллокаций был реализован механизм переиспользования буфера с помощью спецификатора `thread_local` (в том числе с заделом под будущую параллелизацию).

Это позволило сохранить и повторно использовать память между рекурсивными вызовами на всём протяжении работы алгоритма, сократив время работы до ≈ 2.8 секунд.

3.2.3. Сплющивание вектора изоморфизмов (плоский буфер)

Двумерный вектор `std::vector<std::vector<vertex_t>>` приводил к фрагментации памяти, так как каждый внутренний вектор размещался в отдельном блоке кучи. Для достижения непрерывного размещения данных в памяти была разработана структура плоского хранения:

- Все вершины всех найденных изоморфизмов стали размещаться последовательно в одном сплошном одномерном массиве.
- Границы и размеры отдельных изоморфизмов отслеживаются с помощью вспомогательного массива смещений.
- Доступ к отдельному изоморфизму реализован через легковесный прокси-объект, содержащий лишь указатель на начало диапазона в общем буфере и его длину.
- Добавление новых изоморфизмов выполняется прямой записью данных в конец непрерывного буфера.

В сочетании со спецификатором `thread_local` плоская структура обеспечила последовательное размещение данных в кэше процессора и минимизировала системные накладные расходы. В результате время выполнения на графе `mutag_graph` сократилось до 1.8 секунд.

Сводные результаты этапов оптимизации приведены на рисунке 1.

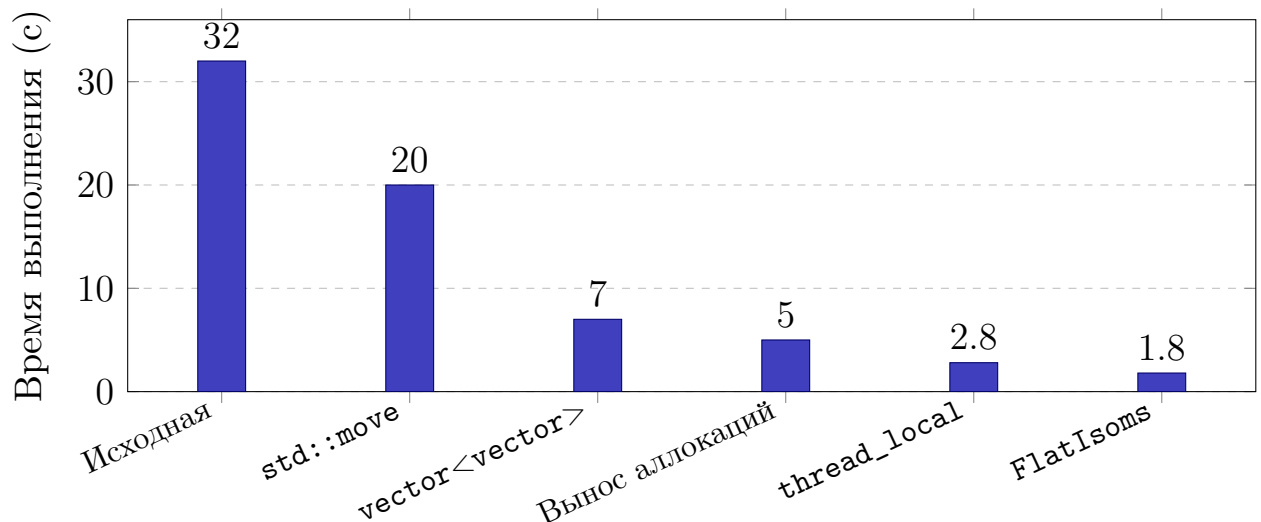


Рис. 1: Динамика снижения времени выполнения по ключевым оптимизациям

3.3. Алгоритмические оптимизации и параллелизация

После локальной оптимизации было решено изучить существующие открытые реализации `gSpan` на C++ для поиска дополнительных способов ускорения. Оригинальная реализация³ от авторов алгоритма устарела и не использует возможности современного C++, а среди доступных на GitHub проектов выбор пал на **gBolt**⁴. Этот проект позиционируется авторами как эффективная по памяти реализация с ускорением вплоть до 100 раз относительно оригинального кода. Ключевые решения `gBolt` были взяты за основу и адаптированы для `Desbordante`.

³<https://www.cs.ucsb.edu/~xian/software/gSpan.htm>

⁴<https://github.com/Jokeren/gBolt>

3.3.1. Оптимизации на основе gBolt

Из gBolt были заимствованы следующие идеи:

- **Механизм проекций.** Проекция хранит точные места, где текущий подграф встречается в коллекции (инвертированный индекс). Главный плюс в том, что при добавлении нового ребра алгоритму не нужно каждый раз заново сканировать исходные графы. Он работает только с уже готовой проекцией: новые варианты ищутся исключительно среди тех рёбер, которые соседствуют с найденными ранее вхождениями. При переходе на следующий шаг рекурсии дочерние проекции создаются только для тех графов, где подграф действительно удалось продолжить. В итоге мы отсекаем огромный объём лишних вычислений и сужаем область поиска до минимума.
- **История обхода (History).** Чтобы быстрее работать с вхождениями внутри проекции используется вспомогательный класс `History`. Он запоминает текущий путь, сохраняя массивы с флагами посещённых рёбер и счётчики вершин. Это позволяет мгновенно проверить, использовалось ли уже конкретное ребро, исключая повторный выбор одних и тех же рёбер и избавляя от необходимости сканировать подграф целиком.
- **Частичное отсечение (Partial Pruning).** Этот механизм отбрасывает неоптимальные пути ещё на этапе генерации новых рёбер, до запуска проверки на каноничность. В его основе лежит строгий контроль за лексикографическим порядком обхода графа. Во-первых, алгоритм прерывает поиск, если присоединяемая вершина имеет метку меньше стартовой (поскольку при обходе этих же рёбер в другом порядке, начав с этой меньшей вершины, мы получим тот же самый подграф). Во-вторых, он следит за тем, чтобы соседние ветви, исходящие из одного узла, обходились строго по возрастанию их меток (нарушение этого порядка также ведет к дублированию). В-третьих, алгоритм отбрасывает по-

пытки добавить прямое (forward) ребро к новой вершине, если из текущей позиции существовала возможность провести обратное (backward) ребро к одной из уже посещённых вершин. Поскольку в лексикографическом порядке gSpan обратные рёбра должны предшествовать прямым, пропуск такого ребра указывает на то, что генерируемый маршрут не является минимальным, и, как следствие, является дубликатом.

- **Оптимизация проверки минимальности (MinGraph).** В исходной реализации метод `IsCanonical` при каждой проверке создавал с нуля полноценный граф Boost (`CreateGraphFromDFSCode`) и на каждом ребре выделял память для сбора всех возможных продолжений.

В новой версии проверка минимальности была полностью переписана по проекционному принципу:

- Локально для этого метода вместо создания графа Boost используется легковесная структура `MinGraph`, буферы которой переиспользуются между вызовами. При этом для хранения самой коллекции графов по-прежнему используются структуры Boost.
- Вместо генерации всех продолжений формируется компактная проекция минимальных путей (`min_projection`), а отслеживание пройденных рёбер выполняется через класс `History` за $O(1)$.
- Проверка разделена на анализ прямых (`IsForwardMin`) и обратных (`IsBackwardMin`) рёбер. Правило приоритета обратных рёбер позволяет мгновенно прерывать проверку (`Exists-Backwards`), если из текущей верши можно провести обратное ребро вместо прямого.

3.3.2. Оптимизация памяти проекций через CSR

После внедрения механизма проекций узким местом стало потребление оперативной памяти: проекции хранят миллионы ссылок на рёбра, а в стандартном графе `boost::adjacency_list` дескриптор ребра занимает слишком много байт. При обработке больших графов это приводило к быстрому раздуванию памяти.

Для решения этой проблемы граф был переведён в формат со сжатым строчным хранением (**Compressed Sparse Row, CSR**). В формате CSR граф хранится в виде плоских массивов, за счёт чего размер дескриптора каждого ребра уменьшился до компактного индекса.

3.3.3. Многопоточность через пул потоков

Дерево поиска частых подграфов часто получается асимметричным, а его глубина непредсказуема. По этой причине банальная параллелизация на уровне начальных семян (стартовых рёбер) неоптимальна: один вычислительный поток может получить глубокую ветвь и работать долгое время, тогда как остальные быстро завершат свои задачи и будут простаивать. Чтобы обеспечить равномерную загрузку ядер процессора, архитектура была перестроена так, чтобы независимые подзадачи генерировались динамически на каждом шаге ветвления.

В gBolt для распараллеливания применялась библиотека OpenMP, однако во избежание внедрения внешних зависимостей в ядро Desbordante от этого подхода решено было отказаться. Вместо этого был реализован собственный пул потоков (`ThreadPool`) исключительно на стандартных инструментах `std::thread`.

Стоит отметить, что в кодовой базе Desbordante уже был реализован собственный пул потоков, однако он не подошёл под специфику алгоритма gSpan. Имеющийся пул использует классическую очередь FIFO, тогда как для эффективного поиска подграфов критически необходима организация очереди задач по принципу стека (LIFO).

Каждое расширение подграфа порождает отдельную задачу. Потоки забирают элементы с вершины стека и продолжают спуск по дереву,

сохраняя естественную семантику поиска в глубину (DFS). Использование стандартного FIFO из существующего пула означало бы переход к поиску в ширину (BFS), что потребовало бы одновременного хранения проекций для всего горизонта дерева и быстро привело бы к исчерпанию оперативной памяти. По этой причине для gSpan был разработан отдельный специализированный `ThreadPool`.

Для безопасной работы потоков логика поиска была изолирована в отдельный класс `SubgraphMiner`. Пул потоков создаёт по одному экземпляру этого класса на каждый аппаратный поток процессора. В процессе работы каждый поток взаимодействует исключительно со своим личным экземпляром майнера, переиспользуя однажды выделенные буферы памяти между задачами и сбрасывая лишь их логическое состояние.

3.3.4. Дополнительные оптимизации

В ходе код-ревью реализации пула потоков было замечено, что задачи создавались для каждого найденного расширения подграфа, из-за чего многие из них сразу же завершались из-за недостаточной поддержки. Создание таких холостых задач приводило к лишним блокировкам мьютексов (при добавлении и извлечении из очереди), аллокациям `std::function` и копированию данных. В результате проверка поддержки была вынесена до создания задачи в пуле, а сам её подсчёт стал выполняться «на лету» прямо во время построения проекции.

Кроме того, в ревью было предложено ограничить порождение параллельных задач по глубине дерева и минимальному размеру проекции. Идея заключалась в том, чтобы параллелить только верхние уровни дерева поиска, а нижние выполнять последовательно в том же потоке, поскольку на глубоких уровнях полезная работа может занимать меньше времени, чем накладные расходы на создание задачи и захват мьютекса. Однако серия замеров и эмпирический подбор параметров выявили компромисс: ограничение параллелизации действительно сокращало расход памяти, но заметно снижало общую скорость работы,

поэтому от этой идеи было решено отказаться.

Финальной рекомендацией стала замена типов индексов вершин и рёбер в `boost::compressed_sparse_row_graph` на `uint32_t` вместо используемого по умолчанию `size_t`. Такой выбор типов уменьшил размер дескрипторов в памяти, сократил размер сохраняемых в проекциях рёбер и сделал граф более компактным для кэша процессора.

4. Эксперимент

Для оценки эффективности оптимизаций была проведена серия замеров производительности. Все тестовые наборы данных и значения порога минимальной поддержки min_sup были взяты из предыдущей работы [6]. Характеристики датасетов приведены в таблице 1.

Таблица 1: Характеристики тестовых датасетов

Датасет	Графов	Вершин (в ср.)	Рёбер (в ср.)
Mutag	188	17.93	19.79
Enzymes	600	32.63	62.14
DandD	1178	284.31	715.66

Эксперименты проводились на тестовом стенде со следующей аппаратной и программной конфигурацией: Intel(R) Core(TM) i5-11260H (6 ядер, 12 потоков, до 4.4GHz), 16 GB RAM, Ubuntu 20.04, g++ 11.2.0, boost 1.85.0. Результаты замеров усреднялись по пяти запускам.

4.1. Однопоточное сравнение

Таблица 2: Сравнение алгоритмов в однопоточном режиме (в секундах)

Датасет	minsup	Desbordante (исх.)	SPMF	gBolt	Desbordante (оптим.)
Mutag	0.6	12	4	0.23	0.23
Mutag	0.4	89	31	0.68	0.68
Mutag	0.2	1198	330	3.32	3.16
Enzymes	0.8	11	9	1.76	1.82
Enzymes	0.7	102	80	13.16	13.34
Enzymes	0.6	984	691	102.02	99.88
DandD	0.6	6	6	1.06	1.75
DandD	0.4	17	18	2.48	2.98
DandD	0.2	89	95	9.02	9.16

Как видно из таблицы 2, оптимизированная реализация gSpan в Desbordante продемонстрировала колоссальный прирост производительности по сравнению с исходной версией на всех тестовых наборах данных.

В целом по всем проведённым тестам ускорение относительно исходной реализации составило от 3.4 до 379 раз при медианном значении 9.7 раза. Наибольший относительный выигрыш зафиксирован на датасете *Mutag* при пороге $min_sup = 0.2$, где время работы сократилось с 1198 секунд до 3.16 секунды (ускорение почти в 380 раз). На графах со сложной структурой связей (*Enzymes* и *DandD*) алгоритм продемонстрировал стабильное ускорение от 3.4 до 9.8 раз.

Кроме того, новая версия *Desbordante* полностью ликвидировала отставание от библиотеки *SPMF*, опережая её в разы во всех сценариях (например, 99.88 с против 691 с на *Enzymes* при $min_sup = 0.6$). Время выполнения вплотную приблизилось к показателям утилиты *gBolt*, а в ряде случаев даже превзошло их: на датасете *Mutag* оптимизированная версия показывает идентичное или лучшее время, на *Enzymes* отставание не превышает 1.4–3.5% (при этом на $min_sup = 0.6$ обходя *gBolt*), а на *DandD* абсолютная разница во времени работы теперь укладывается в узкий диапазон от 0.14 до 0.69 секунды.

4.2. Исследование многопоточности

Поскольку при высоких значениях порога поддержки алгоритм обрабатывает за доли секунды, исследование многопоточного режима проводилось на наиболее ресурсоёмких сценариях (с минимальным значением min_sup).

Таблица 3: Сравнение времени работы в однопоточном и многопоточном режимах (в секундах)

Датасет	minsup	1 поток		12 потоков	
		gBolt	Desbordante	gBolt	Desbordante
Mutag	0.2	3.32	3.16	0.64	0.56
Enzymes	0.6	102.02	99.88	25.77	25.39
DandD	0.2	9.02	9.16	2.07	2.24

Использование многопоточности позволило существенно сократить время вычислений во всех тестовых сценариях (таблица 3): прирост

производительности Desbordante при переходе от 1 к 12 потокам составил от 3.9 до 5.6 раза.

Наибольшее относительное ускорение зафиксировано на датасете **Mutag**: время счёта сократилось в 5.6 раза (с 3.16 до 0.56 с). При этом оптимизированная версия Desbordante уверенно превзошла утилиту gBolt в многопоточном режиме (0.56 с против 0.64 с). На крупном графе **DandD** время выполнения сократилось в 4.1 раза (с 9.16 до 2.24 с), а абсолютное отставание от gBolt сократилось до минимума и составляет всего 0.17 с.

На наиболее трудоёмком датасете **Enzymes** параллелизация снизила время работы Desbordante почти в 3.9 раза (с 99.88 до 25.39 с). Утилита gBolt на этом датасете демонстрирует сопоставимое относительное масштабирование (четырёхкратное ускорение), однако в абсолютных значениях времени выполнения Desbordante показывает лучший результат (25.39 с против 25.77 с у gBolt).

4.3. Потребление памяти

Помимо измерения времени выполнения, также было зафиксировано пиковое потребление памяти при разном количестве потоков.

Таблица 4: Сравнение пикового потребления памяти (в МБ)

Датасет	minsup	1 поток		12 потоков	
		gBolt	Desbordante	gBolt	Desbordante
Mutag	0.2	60.9	55.37	108.26	209.95
Enzymes	0.6	1374.48	1359.39	5351.29	6005.82
DandD	0.2	110.86	332.39	171.23	404.18

Как видно из таблицы 4, в однопоточном режиме алгоритм в Desbordante демонстрирует сопоставимое или даже меньшее потребление памяти по сравнению с gBolt на датасетах **Mutag** (55.37 против 60.9 МБ) и **Enzymes** (1359.39 против 1374.48 МБ). Однако на графе **DandD** расход памяти в Desbordante оказывается выше ровно в 3 раза (332.39 против 110.86 МБ).

При переходе к 12 потокам потребление памяти в Desbordante возрастает сильнее, чем у конкурента, и превышает показатели gBolt во всех сценариях: на Mutag разрыв составляет почти 2 раза, на DandD — в 2.36 раза, а пиковый объём на наиболее ресурсоёмком датасете Enzymes достигает 6.01 ГБ (6005.82 МБ) у Desbordante против 5.35 ГБ у gBolt (отставание около 12.2%).

Повышенное потребление памяти, которое особенно заметно на датасете DandD и в многопоточном режиме, обусловлено особенностями хранения проекций. Каждая запись проекции содержит дескриптор ребра, который из-за использования графовых абстракций библиотеки Boost в Desbordante занимает больше байт, чем легковесные структуры в gBolt. При миллионах записей в проекциях эта разница аккумулируется. В 12-поточном режиме этот эффект кратно усиливается из-за необходимости одновременного выделения памяти под независимые наборы проекций для каждого активного потока.

Дополнительным фактором, увеличивающим расход памяти по сравнению с gBolt, является архитектура самого Desbordante. В памяти осуществляется дублированное хранение загруженной коллекции графов, необходимое для обеспечения перезапуска алгоритма с различными параметрами (например, с другим значением порога поддержки) в рамках одной сессии, что позволяет избежать накладных расходов на повторное чтение и парсинг исходных данных.

4.4. Накладные расходы Python

В отличие от утилиты gBolt, которая является отдельной консольной C++ программой, библиотека Desbordante на практике используется преимущественно через интерфейс Python. Напрямую сравнивать консольную утилиту с вызовами из Python-скриптов не вполне корректно из-за принципиально разных сценариев применения. Поэтому важно оценить, какие реальные накладные расходы привносят привязки `pybind11` по сравнению с прямым запуском алгоритма на C++.

Для получения статистически надёжных результатов каждый тест

запускался по 30 раз. Все результаты представлены в формате: среднее $\pm$ 95% доверительный интервал, а проверка значимости различий между версиями на C++ и Python выполнялась с помощью t -критерия Уэлча. Результаты сравнения времени сведены в таблицу 5, а потребления памяти — в таблицу 6.

Таблица 5: Сравнение времени выполнения на C++ и Python (в секундах)

Датасет	minsup	Потоки	Время C++	Время Python	p -value
Mutag	0.2	1	3.33 ± 0.02	3.40 ± 0.02	< 0.01
Mutag	0.2	12	0.616 ± 0.008	0.639 ± 0.005	< 0.01
Enzymes	0.6	1	106.16 ± 0.18	107.3 ± 0.4	< 0.01
Enzymes	0.6	12	29.3 ± 0.4	29.59 ± 0.04	0.14
DandD	0.2	1	9.87 ± 0.16	10.17 ± 0.13	< 0.01
DandD	0.2	12	2.536 ± 0.009	2.628 ± 0.008	< 0.01

Таблица 6: Сравнение пикового потребления памяти на C++ и Python (в МБ)

Датасет	minsup	Потоки	Память C++	Память Python	p -value
Mutag	0.2	1	55.37 ± 0.02	82.00 ± 0.02	< 0.01
Mutag	0.2	12	209 ± 5	240 ± 5	< 0.01
Enzymes	0.6	1	$1\,359.39 \pm 0.03$	$1\,400.34 \pm 0.04$	< 0.01
Enzymes	0.6	12	$6\,020 \pm 130$	$5\,970 \pm 160$	0.61
DandD	0.2	1	332.30 ± 0.03	362.01 ± 0.03	< 0.01
DandD	0.2	12	399 ± 3	425 ± 2	< 0.01

По времени выполнения (таблица 5) версия на C++ статистически значимо быстрее в пяти сценариях из шести ($p < 0.05$), однако абсолютная разница невелика и во всех случаях не превышает 0.3 секунды. При этом на тесте **Enzymes** в 12 потоков статистически значимой разницы между C++ и Python не зафиксировано вовсе ($p = 0.14$).

По потреблению оперативной памяти (таблица 6) запуск через Python требует на 25–40 МБ больше в однопоточном режиме и на 26–31 МБ больше при 12 потоках на датасетах **Mutag** и **DandD**. На датасете **Enzymes** при 12 потоках статистически значимой разницы в потреблении памяти между C++ и Python не обнаружено ($p = 0.61$).

Полученные замеры показывают, что вызов алгоритма через Python-интерфейс сохраняет высокую скорость вычислений и не приводит к

существенному росту расхода оперативной памяти.

Заключение

В ходе работы были успешно выполнены следующие задачи:

- проведено профилирование исходной версии алгоритма gSpan, определены и ликвидированы основные узкие места;
- исследованы сторонние оптимизированные решения, а ключевые подходы перенесены в проект Desbordante;
- выполнен сравнительный анализ производительности оптимизированного алгоритма с предыдущей версией.

Исходный код всех проведённых изменений доступен на GitHub⁵.

⁵<https://github.com/Desbordante/desbordante-core/pull/777>

Список литературы

- [1] Allamanis Miltiadis, Brockschmidt Marc, Khademi Mahmoud. Learning to Represent Programs with Graphs. — 2018. — [1711.00740](#).
- [2] [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: <https://doi.org/10.1145/3703323.3703725>.
- [3] Leskovec Jure, Kleinberg Jon, Faloutsos Christos. [Graphs over time: densification laws, shrinking diameters and possible explanations](#) // Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery in Data Mining. — KDD '05. — New York, NY, USA : Association for Computing Machinery, 2005. — P. 177–187. — URL: <https://doi.org/10.1145/1081870.1081893>.
- [4] Gilmer Justin, Schoenholz Samuel S., Riley Patrick F. et al. Neural Message Passing for Quantum Chemistry. — 2017. — [1704.01212](#).
- [5] Yan Xifeng, Han Jiawei. gSpan: Graph-Based Substructure Pattern Mining // Proceedings of the 2002 IEEE International Conference on Data Mining. — ICDM '02. — USA : IEEE Computer Society, 2002. — P. 721.
- [6] Максим Малышев. Реализация поиска часто встречающихся подграфов с помощью алгоритма gSpan в Desbordante. — 2025. — URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/gSpan-MalyshevMaxim-2025autumn.pdf>.